IRIUS

gastronomia

Guia do App
Como tudo funciona

Uma aula do zero, de cabo a rabo, para quem conhece React, JavaScript, HTML e CSS, mas nunca viu SQL

Cardápio digital · Room Service · Painel do restaurante

Versão de 22 de outubro de 2026

Antes de começar

Este guia explica **tudo** o que existe no app hoje: o que cada parte faz, por que ela foi feita desse jeito e como as partes conversam entre si. Ele foi escrito para você ler com calma, de preferência na ordem. Não é preciso decorar nada. A ideia é que, no final, você consiga olhar para qualquer arquivo do projeto e saber "ah, isso aqui serve para tal coisa".

Você já conhece a parte de React. Por isso, sempre que aparecer algo novo (banco de dados, SQL, segurança), vou comparar com coisas que você já sabe de JavaScript.

As caixas que aparecem no texto

Analogia

Uma comparação com o mundo real para a ideia "fazer sentido" antes do detalhe técnico.

Para lembrar

O resumo do que importa. Se tiver pouco tempo, leia só essas caixas.

Atenção

Algo que pode dar problema ou que tem a ver com segurança e dinheiro.

Na prática

Algo para você fazer ou testar com as próprias mãos.

Sumário

Parte I · Visão geral

1 O que o app faz hoje

2 A grande mudança: de site estático para app com banco

3 O mapa do sistema

Parte II · O banco de dados, do zero

4 Banco de dados para quem sabe JavaScript

5 As tabelas do Irius, uma por uma

6 A vida de um pedido (os status)

7 Segurança: GRANT, RLS e por que o front não protege nada

8 Funções dentro do banco (RPC)

9 Login, fotos e tempo real: os outros serviços do Supabase

10 Os arquivos SQL do projeto e quando rodar cada um

Parte III · O front-end

11 O mapa das pastas

12 Como o app se conecta ao Supabase

13 O cardápio público

14 O carrinho e o checkout do hóspede

15 O painel: login e gestão do cardápio

16 O painel: pedidos ao vivo, som e cupom de 80mm

Parte IV · Na prática

17 Por que hoje aparece "Cardápio indisponível"

18 Passo a passo: colocando o teste real para rodar

19 Como investigar quando algo dá errado

20 As fotos do cardápio

21 Antes de abrir para hóspedes de verdade

22 Glossário

23 Exercícios para fixar

PARTE I

Visão geral

O que o app faz, o que mudou e quais são as peças do sistema.

1 O que o app faz hoje

O app atende três tipos de pessoa. Pensar neles ajuda a entender por que cada parte existe.

Quem	Onde usa	O que faz
Hóspede (ou cliente)	O cardápio, no celular: `/RestMenu/`	Vê o cardápio em PT, EN ou ES, monta um pedido, informa quarto e dados, escolhe como pagar e acompanha o andamento.
Recepção e cozinha	O painel: `/RestMenu/admin`, aba "Pedidos ao Vivo"	Recebe os pedidos na hora, com um som de alerta. Confere os pedidos "faturar no quarto" ou "dinheiro", envia para a cozinha e imprime o cupom.
Dono (Cristiano)	O painel, aba "Gestão de Cardápio"	Pausa pratos em falta, muda preços, cria e edita itens e sobe fotos, sem precisar de programador.

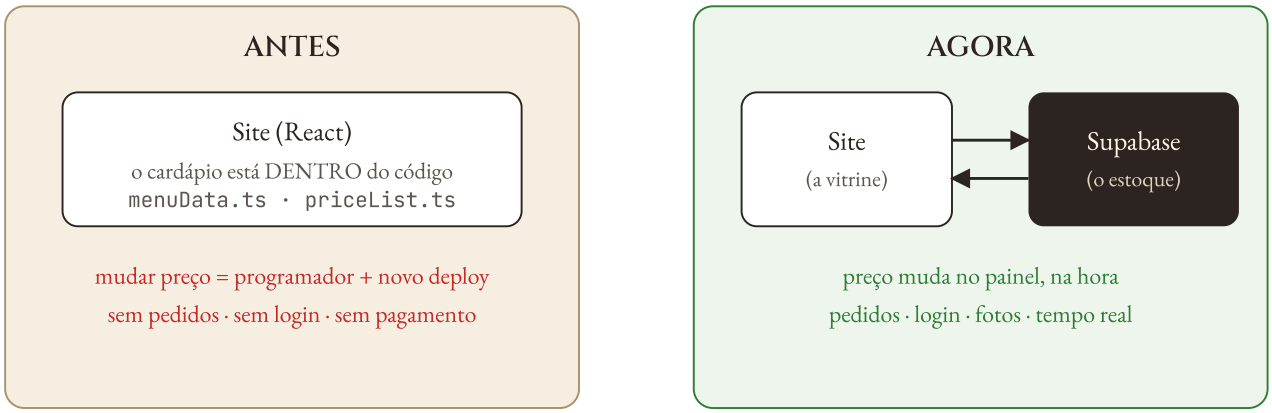
O objetivo de negócio por trás de tudo isso são as duas dores que o Cristiano descreveu:

1. **Menos telefone tocando.** O pedido chega escrito e direto na tela e na impressora, sem o cozinheiro largar o fogão para atender.
2. **Nada sem pagar.** Pedidos PIX ou cartão só vão para a cozinha depois de pagos. "Faturar no quarto" e "dinheiro" passam antes pela conferência da recepção.

2 A grande mudança: de site estático para app com banco

Antes, o cardápio estava **escrito dentro do código**, nos arquivos `menuData.ts` e `priceList.ts`. Mudar o preço da costela era assim: abrir o VS Code, editar, gerar o build e publicar de novo. E não existia pedido, pagamento nem login.

Agora o cardápio, os pedidos e os usuários ficam num **banco de dados na internet** (o Supabase). O site virou só a "vitrine": quando abre, ele **pergunta** ao banco o que mostrar.

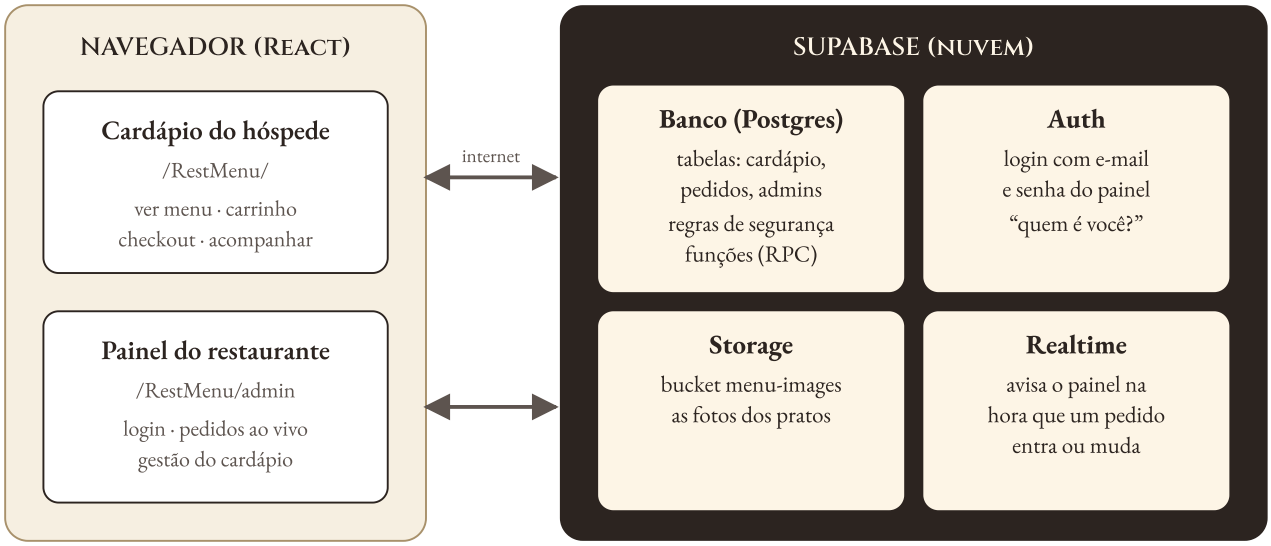


Antes os dados moravam no código. Agora moram no banco, e o site só os mostra.

> **Analogia**
> Antes, o cardápio era **impresso**: para mudar um preço, era preciso mandar imprimir tudo de novo. Agora é como uma **TV de cardápio** ligada a um computador. Muda no computador e todas as TVs mostram o novo preço na hora.

3 O mapa do sistema

Existem duas grandes peças: o **front-end** (o React que você conhece, rodando no navegador) e o **Supabase** (um serviço na nuvem). O Supabase, por sua vez, é um pacote com quatro serviços dentro:



As duas telas do app (à esquerda) e os quatro serviços do Supabase (à direita).

Serviço	Para que usamos
Banco (Postgres)	Guarda categorias, pratos, pedidos e a lista de administradores. Também guarda as **regras de segurança** e as **funções** que calculam o pedido.
Auth	O login do painel. Ele confere e-mail e senha e entrega ao navegador um "crachá" que prova quem é a pessoa.
Storage	Um "Google Drive" para as fotos dos pratos, no bucket `menu-images`.
Realtime	Avisa o painel no instante em que um pedido é criado ou muda de status. É isso que faz o som tocar sem ninguém apertar F5.

Analogia

Pense no Supabase como um **prédio de serviços**. No térreo fica o arquivo (o banco). Na portaria, o segurança que confere crachás (Auth). No subsolo, o depósito de fotos (Storage). E no último andar, um alto-falante que anuncia "chegou pedido!" (Realtime). O seu React é o cliente que entra no prédio e pede coisas.

Para lembrar

Não existe um "servidor nosso" (Node, Express...). O navegador conversa **direto** com o Supabase. É por isso que a segurança precisa morar **dentro do banco** (capítulo 7).

PARTE II

O banco de dados, do zero

SQL explicado para quem só conhece JavaScript: tabelas, segurança e as funções que protegem o dinheiro do restaurante.

4 Banco de dados para quem sabe JavaScript

Tabela = um array de objetos com formato fixo

Você já trabalhou com algo assim em JavaScript:

```
const menu_items = [
  { id: 1, name: "Dadinhos de Coalho", price: 24.00, is_available: true },
  { id: 2, name: "Costela 12h",        price: 92.00, is_available: false },
];
```

Uma **tabela** de banco de dados é exatamente isso, com três diferenças importantes:

1. **Ela mora num servidor**, não na memória do navegador. Todos os celulares veem os mesmos dados.
2. **O formato é obrigatório.** Toda linha tem as mesmas colunas, cada uma com um tipo (texto, número...). Se você tentar salvar `price: "banana"`, o banco recusa.
3. **Ela tem regras.** Por exemplo: "o preço não pode ser negativo" ou "só admin pode alterar".

Cada objeto do array é uma **linha** (ou registro). Cada propriedade é uma **coluna** (ou campo).

SQL: a linguagem para conversar com o banco

SQL é uma linguagem em que você **descreve o que quer**, e o banco descobre como fazer. Compare com o JavaScript:

Em JavaScript (com um array)	Em SQL (com uma tabela)
`items.filter(i ⇒ i.price > 50)`	`select * from menu_items where price > 50;`
`items.map(i ⇒ i.name)`	`select name from menu_items;`
`items.sort((a,b) ⇒ a.price - b.price)`	`select * from menu_items order by price;`
`items.slice(0, 5)`	`select * from menu_items limit 5;`
`items.length`	`select count(*) from menu_items;`
`items.push({ name: "Novo", price: 10 })`	`insert into menu_items (name, price) values ('Novo', 10);`
`items.forEach(i ⇒ { if (i.id ⩵ 2) i.price = 95 })`	`update menu_items set price = 95 where id = 2;`
`items = items.filter(i ⇒ i.id ≠ 2)`	`delete from menu_items where id = 2;`

> **Atenção**
>
> Num `update` ou `delete`, **sempre** coloque o `where`. Sem ele, o comando vale para **todas as linhas da tabela**. Um `update menu_items set price = 95;` deixa o cardápio inteiro custando R$ 95. No SQL não existe Ctrl+Z.

Detalhes de escrita: textos usam **aspas simples** (`'Costela'`), o comando termina com `;` e maiúsculas e minúsculas nos comandos tanto faz (`SELECT` = `select`).

Juntando tabelas (JOIN)

O prato guarda só o **número** da categoria (`category_id`), não o nome. Para ver os dois juntos, você "junta" as tabelas:

```
select menu_items.name, categories.name as categoria
from menu_items
join categories on categories.id = menu_items.category_id;
```

Em JavaScript seria um `items.map(i ⇒ ({ ...i, categoria: categories.find(c ⇒ c.id ⩵ i.category_id).name }))`.

Os tipos de dado que usamos

Tipo SQL	Parecido com	Onde usamos e por quê
`text`	string	Nomes, descrições, quarto.
`int` / `bigint`	number (inteiro)	Quantidade, ordem de exibição, IDs.

`numeric(10,2)`	number com 2 casas	**Dinheiro.** Guarda o valor exato. Em JS, `0.1 + 0.2` dá `0.30000000000000004`; o `numeric` não erra centavos.
`boolean`	true/false	`is_available`, `allow_room_charge`.
`uuid`	string aleatória	ID do pedido, por exemplo `8368db83-…`. É impossível de adivinhar e por isso serve como "senha" do pedido.
`timestamptz`	Date	Data e hora com fuso, como `created_at`.
`jsonb`	um objeto JS	`translations`: `{"en": {"name": "Ribs"}, "es": {...}}`.

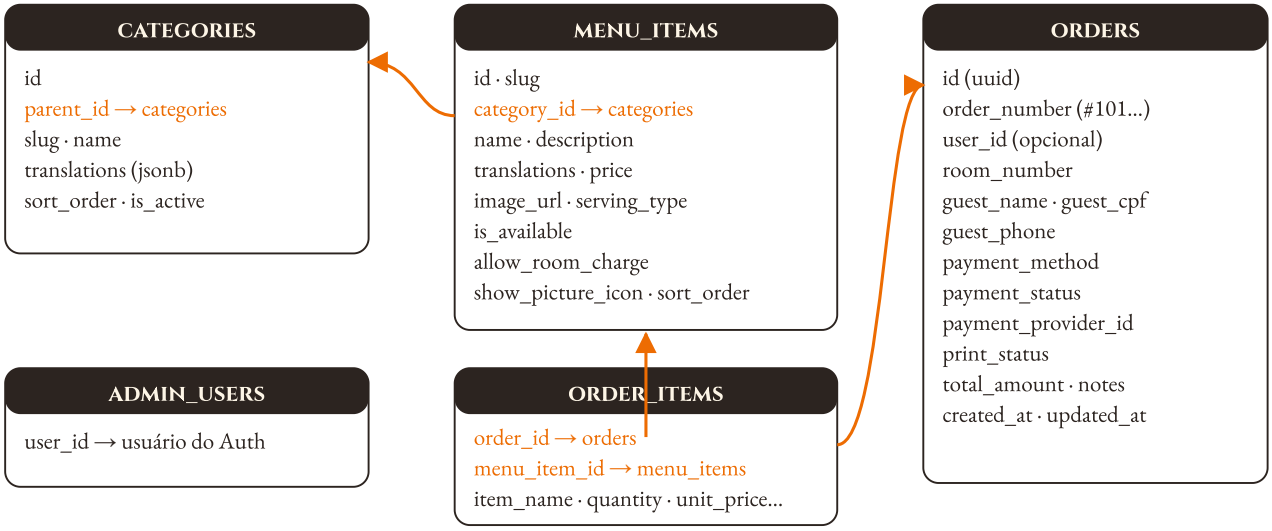
Chaves e regras

- **Chave primária (`primary key`)**: a coluna que identifica a linha de forma única, como o `key` de uma lista no React. Em todas as nossas tabelas é o `id`.
- **Chave estrangeira (`references`)**: uma coluna que aponta para o `id` de outra tabela. O `menu_items.category_id` aponta para `categories.id`, e o banco **não deixa** você colocar um prato numa categoria que não existe.
- `not null`: o campo é obrigatório.
- `unique`: o valor não pode se repetir (o `slug`, por exemplo).
- `check (...)`: uma validação, como `check (price ⩾ 0)` ou `check (quantity between 1 and 20)`.
- `default`: o valor automático quando você não informa nada. `created_at default now()` grava a hora atual.
- `generated ... as identity`: um contador automático (1, 2, 3...). É assim que os pedidos ganham os números #101, #102...

> **Para lembrar**
>
> Essas regras ficam no banco. Mesmo que alguém burle o seu formulário React, o banco recusa um dado inválido. A validação do front serve para a **experiência** do usuário (mensagens bonitas); a do banco serve para a **segurança**.

5 As tabelas do Irius, uma por uma

Todas as tabelas são criadas pelo arquivo `supabase/schema.sql`. Veja como elas se ligam:



As cinco tabelas. As setas laranja são as chaves estrangeiras: "esta coluna aponta para aquela tabela".

CATEGORIES: o cardápio em árvore

O cardápio antigo tinha três níveis: aba (Vinhos), seção (Vinhos Tintos) e subseção (Malbec). Em vez de criar três tabelas, cada categoria pode ter um **pai**, pela coluna `parent_id`:

id	parent_id	slug	name	O que é
6	*null*	vinho	Vinhos	aba (não tem pai)
24	6	vinhos_tintos	Vinhos Tintos	seção (o pai é a aba 6)
32	24	malbec	Malbec	subseção (o pai é a seção 24)

Outras colunas: `slug` é um apelido fixo e sem espaços ("vinhos_tintos"), útil para o código achar a categoria sem depender do número. `sort_order` define a ordem de exibição. `is_active` esconde a categoria inteira. `translations` guarda os nomes em inglês e espanhol, e o `name` é sempre o português.

MENU_ITEMS: os pratos e bebidas

Coluna	Para que serve
`category_id`	Em qual categoria o item aparece. Pode ser uma aba, uma seção ou uma subseção.
`slug`	O antigo id do código (`c1`, `v14`...). A cerveja virou `c1_beer` porque o `c1` estava repetido.
`name`, `description`	Em português. É o que sai no cupom da cozinha.
`translations`	`{"en": {"name": "...", "description": "..."}, "es": {...}}`. Se faltar, o app mostra o português.
`price`	O preço oficial. **É o único preço que vale** (capítulo 8).
`image_url`	Caminho da foto: antigo (`lanches/burger-chef.webp`, dentro de `public/`) ou URL completa do Storage.
`is_available`	Desligado = "Indisponível no momento". Continua aparecendo, mas não dá para pedir.
`allow_room_charge`	Se o item pode ser "faturado no quarto". **Hoje está desligado em todos.**
`serving_type`	`glass` (taça, mostra o ícone de taça) ou `bottle` (garrafa).
`show_picture_icon`	Mostra o ícone de câmera ao lado do nome. Liga sozinho quando uma foto é enviada pelo painel.

ORDERS: os pedidos

Uma linha por pedido, com o quarto, nome, CPF, telefone, a forma de pagamento (`pix`, `credit_card`, `room_charge` ou `cash_on_delivery`), o **status do pagamento** (capítulo 6), o **status de impressão** (`pending`, `printed`, `error`), o total e as observações. O `id` é um UUID secreto, enquanto o `order_number` é o número amigável da comanda (#101, #102...). Há ainda o `payment_provider_id`, que vai guardar o código da cobrança no gateway de pagamento real. Hoje ele recebe `SIMULADO-…` quando o pagamento é simulado.

order_items: os itens de cada pedido, como uma "foto"

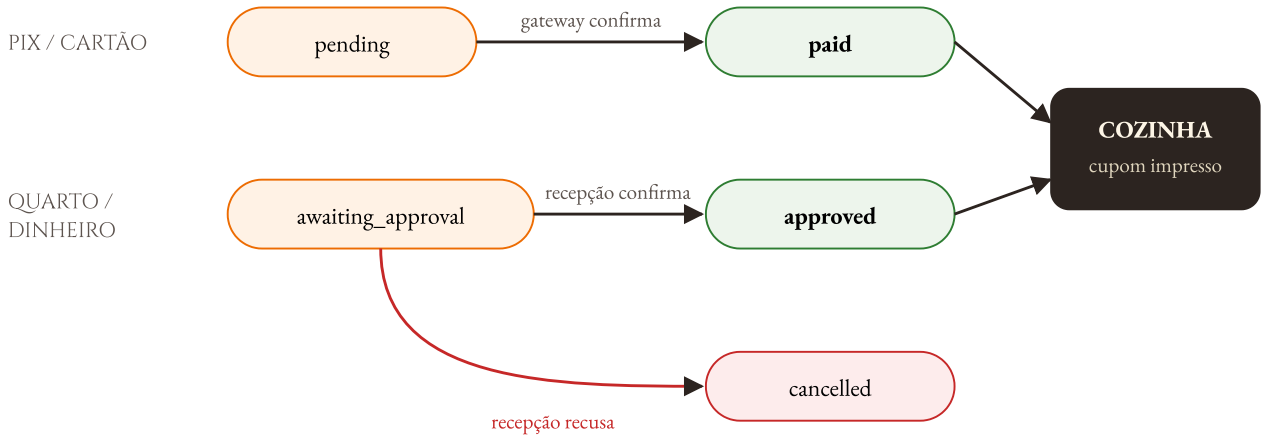
Cada item do pedido guarda o `item_name` e o `unit_price` **do momento da compra**. Se amanhã a costela passar a custar R$ 99, o pedido de ontem continua mostrando R$ 92. É a mesma lógica de uma nota fiscal: ela não muda depois de emitida. A coluna `line_total` (quantidade × preço) é calculada pelo próprio banco.

admin_users: quem manda

Uma lista simples com os IDs dos usuários que são administradores. Estar logado **não basta**: a pessoa precisa estar nessa lista. Isso evita que, no futuro, um hóspede com login vire "admin" sem querer.

6 A vida de um pedido (os status)

A coluna `payment_status` é o coração das regras do hotel. Cada pedido segue um destes caminhos:



*Só pedidos **paid** ou **approved** chegam à cozinha.*

Status	Significado	Quem muda
`pending`	PIX/cartão criado, esperando o pagamento. **Não aparece no painel**.	A função `create_safe_order` cria assim.
`awaiting_approval`	Faturar no quarto ou dinheiro: a recepção precisa conferir.	A função `create_safe_order` cria assim.
`paid`	Pago de verdade.	Hoje, o botão DEV de simulação. No futuro, o gateway de pagamento.
`approved`	Liberado pela recepção, mas o dinheiro ainda vai entrar (no check-out ou na entrega).	Botão "Confirmar Quarto" do painel.
`cancelled`	Recusado.	Botão "Recusar" do painel.

> **Para lembrar**
> "Faturar no quarto" confirmado vira `approved`, e não `paid`, porque ninguém pagou ainda. Assim o relatório nunca mistura "dinheiro no caixa" com "dinheiro a receber".

7 Segurança: GRANT, RLS e por que o front não protege nada

O problema: o seu código React é público

Tudo o que roda no navegador pode ser visto e alterado por quem estiver usando o navegador. Abra o DevTools (F12) em qualquer site: dá para ler o JavaScript, mudar valores e disparar requisições. Se o cardápio confiasse no front para dizer "a costela custa R$ 92", alguém poderia mandar "a costela custa R$ 0,01".

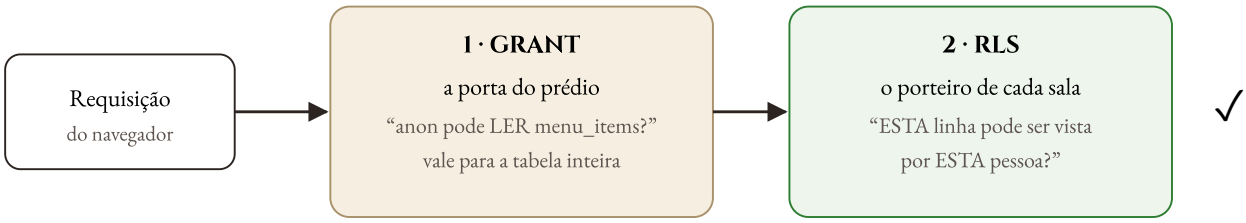
Além disso, a **chave** que o app usa para falar com o Supabase (`VITE_SUPABASE_ANON_KEY`) vai junto no JavaScript publicado. **Isso é normal e esperado**: ela é a chave "publishable" (pública). Ela só abre as portas que o banco permitir.

> **Atenção**
> O Supabase tem também uma chave **secret / service_role**, que ignora todas as regras. Ela **nunca** pode ir para o código do site, nem para arquivo com `VITE_` no nome. Tudo que começa com `VITE_` vai parar no JavaScript que qualquer um baixa.

Os três tipos de visitante

Papel	Quem é
`anon`	Qualquer pessoa sem login: o hóspede no cardápio.
`authenticated`	Alguém que fez login pelo Auth.
admin	Um `authenticated` que está na tabela `admin_users`. A função `is_admin()` responde se a pessoa é admin.

Duas camadas de proteção



Para chegar aos dados, a requisição passa pelas duas camadas.

1. GRANT (permissão por tabela). Diz o que cada papel pode fazer com a tabela *como um todo*. Por exemplo:

```
grant select on public.menu_items to anon, authenticated;           -- todos podem LER
grant insert, update, delete on public.menu_items to authenticated; -- só logados podem ESCREVER
```

Você desligou a opção "Automatically expose new tables" no Supabase. Isso é bom para a segurança, mas significa que, sem esses GRANTs, **ninguém** consegue ler nada. É exatamente o erro que você está vendo hoje (capítulo 17).

2. RLS (Row Level Security, segurança por linha). Depois de passar pela porta, cada linha é conferida por uma **policy** (política). Veja uma delas lida em português:

```
create policy "menu_items_admin_all" on public.menu_items
  for all to authenticated          -- vale para ler/criar/editar/apagar, para quem está logado
  using (public.is_admin())         -- ...mas só se a pessoa for admin
  with check (public.is_admin());   -- e o que ela gravar também passa pela mesma regra
```

É assim que um usuário logado que **não** é admin passa pela porta (GRANT) mas é barrado pelo porteiro (RLS).

Quem pode o quê, hoje

Ação	Hóspede (anon)	Logado comum	Admin
Ver o cardápio (categorias ativas e seus itens, inclusive os pausados)	✓	✓	✓ (inclusive os ocultos)
Criar, editar ou apagar prato e categoria	✗	✗	✓
Criar pedido	Só pela função `create_safe_order`. Ninguém grava direto na tabela.		

Ver pedidos	✗ (só o status do seu, via `get_order_status`)	só os próprios	todos
Mudar status do pedido	✗	✗	✓, mas **só** `payment_status` e `print_status`
Mudar valor, itens ou dados do hóspede	✗	✗	✗ (travado)
Enviar ou apagar fotos	✗	✗	✓

Analogia

O GRANT é o cartão que abre a porta do prédio. O RLS é o porteiro de cada sala, que olha o crachá e uma lista de nomes antes de deixar entrar. Os dois trabalham juntos.

8 Funções dentro do banco (RPC)

Uma **função** no banco funciona como uma função JavaScript que mora no servidor. O app chama pelo nome:

```
const { data, error } = await supabase.rpc("nome_da_funcao", { parametro: valor });
```

"RPC" quer dizer *Remote Procedure Call*: chamar uma função que está em outro computador.

security definer: o "funcionário de confiança"

O hóspede não pode gravar na tabela `orders`. Então como o pedido é criado? Por uma função marcada como `security definer`. Ela roda com a permissão do **dono do banco**, e não com a de quem chamou. Só que ela faz **apenas** o que está escrito nela.

> **Analogia**
> O cliente não entra na cozinha. Ele entrega o pedido ao garçom (a função), que entra na cozinha e faz exatamente o que o procedimento manda. O cliente não pode pedir ao garçom "mude o preço", porque isso não está no procedimento.

create_safe_order: a função que protege o dinheiro

É chamada quando o hóspede clica em "Enviar pedido". O front manda **apenas** o seguinte:

```
supabase.rpc("create_safe_order", {
  p_room_number: "204",
  p_guest_name: "Ana Souza",
  p_payment_method: "pix",
  p_items: [{ menu_item_id: 12, quantity: 2, notes: "sem cebola" }], // SEM preço!
  p_guest_cpf: "52998224725", p_guest_phone: "51999990000", p_notes: "tocar a campainha",
});
```

Dentro do banco, a função segue esta ordem:

1. **Valida o hóspede**: quarto com até 10 caracteres (letras, números, hífen), nome com até 120, CPF com 11 dígitos e telefone com 8 a 15 dígitos (só números). Pontos, traços e espaços são removidos antes.
2. **Escolhe o status inicial**: `pending` (PIX/cartão) ou `awaiting_approval` (quarto/dinheiro).
3. **Valida a lista de itens**: precisa ser uma lista com 1 a 30 linhas, quantidade inteira de 1 a 20 e observação de até 200 caracteres.
4. **Confere cada item no banco**: ele existe? Está disponível? A categoria está visível? Se o pagamento é "faturar no quarto", o item aceita isso?
5. **Busca o preço real** de cada item na tabela e **calcula o total**. O front nunca manda preço.
6. **Grava o pedido e os itens de uma vez só.** Se qualquer passo falhar, nada fica gravado pela metade.
7. **Devolve** o `order_id` (UUID), o número (#101), o total e o status.

> **Para lembrar**
> O total exibido no carrinho é só uma **prévia**. O total que vale é o calculado pelo banco. Se alguém mexer no JavaScript, só consegue estragar a própria prévia.

Quando algo está errado, a função interrompe tudo com uma mensagem ("Um ou mais itens não estão disponíveis no momento."). O front traduz essa mensagem para o idioma do hóspede.

As outras funções

Função	O que faz
`is_admin()`	Responde `true` ou `false`: quem está logado está em `admin_users`? O painel usa isso depois do login, e as regras RLS também.
`is_category_visible(id)`	Sobe pela árvore (subseção → seção → aba) e só responde "visível" se todas estiverem ativas. Desativou "Vinhos"? Tudo dentro some.
`get_order_status(order_id)`	O hóspede não pode ler a tabela `orders`, mas pode perguntar "como está o meu pedido?" passando o UUID. A resposta não traz nome, CPF nem telefone.
`dev_simulate_payment(order_id)`	**Só para testes.** Marca um PIX/cartão pendente como pago, sem dinheiro. Fica em arquivo separado e **precisa ser apagada antes de abrir para hóspedes** (capítulo 21).

9 Login, fotos e tempo real: os outros serviços do Supabase

Auth: o login

Quando o Cristiano digita e-mail e senha no `/admin`, o Auth confere e devolve uma **sessão**: um "crachá" digital (um token chamado JWT) que o supabase-js guarda no navegador e envia, sozinho, em toda requisição seguinte. O banco lê o crachá e sabe quem é a pessoa (`auth.uid()`). Por isso as regras RLS conseguem dizer "só se for admin".

Os usuários do Auth **não ficam nas nossas tabelas**: eles moram numa área do próprio Supabase (*Authentication* → *Users*). A nossa `admin_users` só guarda o ID de quem é admin.

Storage: as fotos

O bucket `menu-images` é público para **ver** (qualquer um abre a foto pelo link) e restrito para **enviar e apagar** (só admin). Ele aceita só PNG, JPEG, WebP e AVIF, com até 5 MB. O SVG ficou de fora porque pode carregar código malicioso. Cada foto recebe uma **URL pública**, que é o que fica gravado em `menu_items.image_url`.

Realtime: o alto-falante

Normalmente o navegador só recebe dados quando pergunta. Com o Realtime, o navegador abre uma **conexão que fica aberta** (um WebSocket) e o Supabase manda um aviso sempre que uma linha de `orders` é criada ou alterada. O painel recebe esse aviso, busca o pedido completo e toca o som. O Realtime também respeita o RLS: só o admin recebe os avisos.

> **Analogia**
>
> Sem Realtime, é como ligar para a recepção a cada minuto perguntando "chegou pedido?". Isso se chama *polling*, e o hóspede usa esse jeito para acompanhar o pedido, a cada 8 segundos. Com Realtime, é a recepção que tem um rádio ligado e **escuta** quando alguém fala.

10 Os arquivos SQL do projeto e quando rodar cada um

SQL não "roda sozinho" como o seu React. Você **cola o conteúdo no SQL Editor do Supabase e clica em Run**. Os arquivos ficam na pasta `supabase/`:

Arquivo	O que faz	Quando rodar
`supabase/schema.sql`	Cria tabelas, regras (GRANT e RLS), funções, Realtime e o bucket de fotos. É a "planta da casa".	Uma vez, e de novo sempre que ele mudar.
`supabase/seed.sql`	Preenche o cardápio com as 37 categorias e os 143 itens que estavam no código antigo, com as traduções.	Uma vez, depois do schema.
`supabase/dev/payment_simulator.sql`	Cria a função do botão DEV de pagamento simulado.	Só para testar.
`supabase/dev/remove_payment_simulator.sql`	Apaga a função acima.	**Antes** de abrir para hóspedes.

"Pode rodar de novo sem quebrar"

Os arquivos foram escritos para poderem ser rodados mais de uma vez. Algumas expressões que garantem isso:

- `create table if not exists`: só cria se ainda não existir.
- `create or replace function`: cria ou atualiza a função.
- `drop policy if exists ...` seguido de `create policy ...`: apaga a regra antiga e recria a nova.
- `on conflict (slug) do nothing` (no seed): se o item já existe, não mexe. Assim o seed **não apaga** o que o dono mudou no painel.

> **Atenção**
>
> O `create table if not exists` **não altera** uma tabela que já existe. Se no futuro precisarmos adicionar uma coluna, será com um comando novo (`alter table ... add column ...`), e não rodando o schema de novo.

O seed foi **gerado por um programa**, que leu os antigos `menuData.ts`, `priceList.ts` e os JSON de tradução. Ninguém digitou os 143 itens à mão. No final, o próprio seed confere se entraram exatamente 37 categorias e 143 itens; se faltar algo, ele desfaz tudo.

PARTE III

O front-end

O React que você já conhece: onde está cada coisa, como ele fala com o Supabase e como funcionam o carrinho, o checkout e o painel.

11 O mapa das pastas

```
irius-gastronomia/
├ .env.local                URL + chave pública do Supabase (não vai pro GitHub)
├ .env.development.local    liga o simulador de pagamento, só no "npm run dev"
├ .env.example              modelo sem as chaves
├ package.json              + @supabase/supabase-js · script "postbuild" (404.html)
├ public/                   imagens antigas (só 3 fotos existem de verdade)
├ supabase/                 os arquivos SQL (capítulo 10)
│  ├ schema.sql · seed.sql
│  └ dev/ payment_simulator.sql · remove_payment_simulator.sql
├ docs/                     este guia
└ src/
   ├ main.tsx               escolhe: cardápio ou painel (/admin)
   ├ App.tsx                a página do cardápio + carrinho + checkout
   ├ vite-env.d.ts          tipos das variáveis VITE_...
   ├ lib/
   │  ├ supabase.ts         a conexão com o Supabase (um arquivo só)
   │  └ menuFormat.ts       idioma, preço em R$ e endereço da foto
   ├ hooks/useMenu.ts       busca o cardápio e monta a árvore
   ├ cart/                  o carrinho (Context)
   │  ├ cartStore.ts · CartProvider.tsx
   ├ checkout/              fluxo do hóspede
   │  ├ CartBar · CheckoutSheet · CartStep · DetailsStep
   │  ├ PaymentStep · StatusStep · checkoutUtils.ts
   ├ components/            as telas do cardápio (Menu, Tabs, MenuItem, Modal...)
   │  ├ MenuSkeleton        novo: carregamento elegante
   │  └ UnavailableTag      novo: selo "Indisponível no momento"
   ├ admin/                 o painel /admin
   │  ├ AdminApp · LoginForm
   │  ├ MenuManager · ItemRow · ItemFormModal · ToggleSwitch
   │  ├ useAdminMenu.ts · adminUtils.ts
   │  └ orders/             pedidos ao vivo
   │     ├ useLiveOrders.ts · alertSound.ts · orderTypes.ts
   │     └ OrdersBoard · ReceiptModal (cupom 80mm)
   └ locales/pt|en|es.json  textos da interface (os do cardápio estão no banco)
```

Foram **apagados** os arquivos `src/data/menuData.ts` e `src/data/priceList.ts`, e dos JSON de tradução saíram os nomes dos pratos e categorias. Tudo isso agora vive no banco.

12 Como o app se conecta ao Supabase

As variáveis de ambiente (.env)

São configurações que ficam **fora do código**. O Vite lê esses arquivos e, para tudo que começa com `VITE_`, deixa o valor disponível em `import.meta.env`:

```
# .env.local  (lido no "npm run dev" E no "npm run build")
VITE_SUPABASE_URL=https://ubwdzhatunywjedkrnju.supabase.co     # sem /rest/v1 no final!
VITE_SUPABASE_ANON_KEY=sb_publishable_...

# .env.development.local  (lido SÓ no "npm run dev")
VITE_PAYMENT_SIMULATOR=true
```

Os dois arquivos terminam em `.local`, e o `.gitignore` ignora `*.local`. Por isso eles nunca vão para o GitHub.

O cliente: src/lib/supabase.ts

```
import { createClient } from "@supabase/supabase-js";
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

Esse objeto `supabase` é importado no resto do app. Pense nele como um `fetch` **turbinado** que já sabe o endereço, já manda a chave e já envia o crachá do login quando ele existe.

Com fetch, "na mão"

```
await fetch(URL + "/rest/v1/menu_items?select=*&order=sort_order",
{
  headers: { apikey: KEY }
}).then(r ⇒ r.json())
```

Com supabase-js

```
await supabase
  .from("menu_items")
  .select("*")
  .order("sort_order")
```

Os dois fazem a **mesma requisição HTTP**. O supabase-js só escreve a URL por você e devolve `{ data, error }`. Os métodos lembram SQL: `.from()` é a tabela, `.select()` as colunas, `.eq("id", 5)` é o `where id = 5`, `.order()` é o `order by`, e `.update()` / `.insert()` mudam dados.

Como o app decide entre cardápio e painel (main.tsx)

O projeto não usa React Router. O `main.tsx` olha o endereço: se terminar em `/admin`, mostra o painel; senão, o cardápio. O painel é carregado com `lazy(() ⇒ import(...))`, isto é, **só quando alguém abre o /admin**. O hóspede não baixa esse código.

O GitHub Pages não sabe que `/RestMenu/admin` é uma "página do React". Se alguém abrir esse endereço direto, ele procura um arquivo `admin` e dá 404. A solução está no `package.json`: o script `postbuild` copia o `index.html` para `404.html`. Quando o GitHub não acha a página, entrega o 404.html, que é o próprio app, e o React resolve o resto.

13 O cardápio público

useMenu: buscar e organizar

O hook `src/hooks/useMenu.ts` faz duas buscas ao mesmo tempo (`Promise.all`): categorias ativas e itens. Depois ele **monta a árvore** e "achata" cada aba numa lista na ordem certa, intercalando títulos e itens, igual ao antigo `menuData.ts`:

```
entriesByTab["vinho"] = [
  { kind: "section",    category: { name: "Vinhos Tintos" } },
  { kind: "subsection", category: { name: "Malbec" } },
  { kind: "item",       item: { name: "Punto Final Gran Reserva", price: 240 } },
  ...
]
```

O hook devolve `{ tabs, entriesByTab, loading, error, reload }`. O `Menu.tsx` só percorre essa lista e decide: título grande, título pequeno ou cartão de item.

> **Para lembrar**
>
> Categorias e itens que são "irmãos" dividem o mesmo `sort_order`. É assim que um item pode aparecer antes de uma seção, se o dono quiser.

menuFormat.ts: três ajudantes

- `localize(item, idioma)`: pega o nome e a descrição em EN/ES de `translations`; se faltar, usa o português.
- `formatPrice(68)`: devolve "R$ 68,00".
- `resolveImageUrl(url)`: se a foto já é um link completo (do Storage), usa direto. Se é um caminho antigo (`lanches/x.webp`), coloca o `/RestMenu/` na frente. É por isso que os caminhos antigos não têm barra no início.

Os componentes

Componente	O que faz agora
`Menu`	Usa o `useMenu`. Enquanto carrega, mostra o `MenuSkeleton`. Se der erro, mostra "Cardápio indisponível" com o botão "Tentar novamente".
`Tabs`	As abas vêm do banco (antes eram uma lista fixa).
`MenuItem`	O cartão do prato, com o botão redondo + (que vira o número de unidades no carrinho). Indisponível: nome apagado, preço riscado, selo e botão desabilitado.
`Modal`	A foto grande, com quantidade (− 1 +), observação e o botão "Adicionar · R$ X". Se a foto não existir, mostra a imagem padrão.
`MenuSkeleton`	Blocos bege com brilho suave no formato do cardápio, para a página não "pular" quando os dados chegam.
`UnavailableTag`	O selo "— *Indisponível no momento* —", traduzido.

Algumas abas (Entradas, Principais, Lanches, Sobremesas) abrem o modal com foto ao clicar no item. Bebidas e Vinhos não abrem, mas têm o botão +.

14 O carrinho e o checkout do hóspede

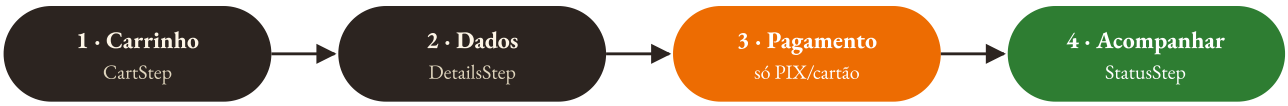
O carrinho é um Context

O carrinho precisa ser lido em vários lugares: o botão + de cada prato, o modal, a barra do rodapé e o checkout. Passar isso por props de componente em componente seria um inferno. Por isso usamos um **Context** do React.

> **Analogia**
>
> Context é um **quadro de avisos na parede da cozinha**. Qualquer pessoa da equipe olha o quadro, e ninguém precisa repassar o recado de mão em mão. O `CartProvider` é quem pendura o quadro (em volta do `App`), e o `useCart()` é olhar para ele.

- `src/cart/cartStore.ts`: define o formato e o hook `useCart()`.
- `src/cart/CartProvider.tsx`: guarda as linhas do carrinho, soma quantidade e subtotal e junta itens iguais (mesmo prato + mesma observação = mesma linha). Ele também **salva no navegador** (`localStorage`, chave `irius:cart`), para o carrinho sobreviver a um recarregamento da página.
- Ele também lembra o **último pedido** enviado (`irius:active-order`, por 8 horas). Assim, se o hóspede fechar a janela, a barra mostra "Acompanhar pedido #101".

As etapas do checkout



A janela `CheckoutSheet` troca de etapa. "Faturar no quarto" e "dinheiro" pulam a etapa 3.

CartBar é a barra escura no rodapé ("Ver pedido · 4 · R$ 283,00"). Ao clicar, abre o **CheckoutSheet**, que mostra uma etapa por vez:

1. **CartStep**: a lista com + e −, lixeira e observação por item.
2. **DetailsStep**: quarto, nome completo, telefone, CPF (ou "não tenho CPF", para estrangeiros), forma de pagamento e observações. Ao enviar, chama `create_safe_order`. O quarto, o nome e o telefone ficam lembrados no aparelho (`irius:guest`); o CPF não, por privacidade.
3. **PaymentStep**: só para PIX/cartão. Mostra um QR Code desenhado (de mentira) e, em modo de teste, o botão `🔨 DEV: Simular Pagamento Aprovado`, que chama `dev_simulate_payment`.
4. **StatusStep**: "Aguardando confirmação da recepção" → "Pedido enviado para a cozinha" → "Em preparo". Pergunta ao banco a cada 8 segundos, via `get_order_status`.

Regras visíveis na tela de dados

- **Faturar no quarto** fica desabilitado se *algum* item do carrinho não aceitar, e a tela mostra quais ("Não disponível para: Suco de Laranja"). É a mesma regra do banco, mostrada antes para o hóspede não levar um erro.
- **PIX e cartão** ficam desabilitados ("Pagamento online em breve") quando o simulador está desligado, porque ainda não existe gateway de verdade.
- O **CPF** é conferido pelo *dígito verificador*: os dois últimos números do CPF são calculados a partir dos 9 primeiros. Se a pessoa errar um número, a conta não fecha e o app avisa.

Por que o botão DEV nunca aparece para hóspedes

```
export const PAYMENT_SIMULATOR =
  import.meta.env.DEV && import.meta.env.VITE_PAYMENT_SIMULATOR === "true";
```

`import.meta.env.DEV` só é verdadeiro no `npm run dev`. No `npm run build` ele vira `false`, e o Vite **remove** o código do botão do arquivo final. Isso foi conferido: o texto do botão não existe no build.

15 O painel: login e gestão do cardápio

A porta de entrada (AdminApp.tsx)

1. Pergunta ao supabase-js se já existe sessão (`getSession`) e passa a escutar mudanças (`onAuthStateChange`: entrou, saiu, crachá renovado).
2. Sem sessão, mostra o `LoginForm` (e-mail, senha, olho para mostrar a senha, erros em português).
3. Com sessão, o `AdminGate` chama `is_admin()`. Se responder `false`, aparece "Esta conta não tem acesso". Se `true`, o painel com as duas abas.

> **Para lembrar**
> Esconder botões no front é só conforto. Quem barra de verdade é o banco: um usuário que não é admin, mesmo que "hackeie" a tela, recebe recusa em cada tentativa de alterar dados.

Gestão de Cardápio (MenuManager)

- **Busca** que ignora acentos ("parmegiana" acha "Parmegiana"), **filtro por aba** e o botão **"Só pausados (N)"**.
- Itens agrupados por categoria, na mesma ordem do cardápio ("Vinhos › Vinhos Tintos › Malbec").
- **ItemRow** (cada linha): miniatura, nome, **preço clicável** (vira um campo; Enter salva, Esc cancela; aceita "68", "68,5", "R$ 1.234,56"), **interruptor** Disponível/Pausado e o botão Editar.

Atualização "otimista"

Ao clicar no interruptor, a tela muda **na hora**, sem esperar o banco responder. Se o banco recusar, o `useAdminMenu` **desfaz** a mudança e mostra o erro. É o que apps como o Instagram fazem com o "curtir": parece instantâneo.

O modal de item (ItemFormModal)

Ele cria ou edita: nome, descrição, categoria (com a hierarquia indentada), preço, disponível e aceita faturar no quarto. Em "Traduções e mais opções" ficam os nomes e descrições em EN/ES e o tipo de serviço (taça/garrafa).

O caminho de uma foto

1. O dono escolhe ou arrasta uma imagem.
2. **O próprio navegador reduz a foto** para no máximo 1600 px e converte para WebP (ou JPEG, se o navegador não gerar WebP), usando um `<canvas>` invisível. Uma foto de celular de 6 MB virou 133 KB no teste.
3. Ela sobe para o bucket com um nome aleatório (`items/uuid.webp`).
4. O Storage devolve a **URL pública**, e o modal mostra a prévia.
5. Só ao clicar em **Salvar** a URL vai para `image_url`, e o ícone de câmera liga.
6. Se o dono **cancelar**, a foto enviada é **apagada** do bucket, para não deixar lixo.

16 O painel: pedidos ao vivo, som e cupom de 80mm

useLiveOrders: o cérebro da aba de pedidos

Fica em `src/admin/orders/useLiveOrders.ts` e faz quatro coisas:

1. **Carga inicial**: busca os pedidos das últimas 24 h com status `awaiting_approval`, `approved` ou `paid`, já com os itens de cada um.
2. **Escuta o Realtime**: a cada aviso de "pedido X mudou", busca o pedido X completo (o aviso não traz os itens) e atualiza a tela.
3. **Reserva**: a cada 30 segundos, busca tudo de novo. Se o Wi-Fi derrubar o Realtime, o painel continua funcionando, e o indicador mostra "Reconectando".
4. **Decide o alerta**: guarda o último status conhecido de cada pedido. Toca o som da **recepção** quando aparece um pedido novo em `awaiting_approval` e o da **cozinha** quando um pedido passa de `pending` para `paid`. Um PIX ainda não pago **não** toca nada.

O hook é usado no `AdminApp`, e não dentro da aba. Por isso os pedidos e o som continuam funcionando mesmo com a aba "Gestão de Cardápio" aberta.

Dois aparelhos ao mesmo tempo

Imagine a recepção e o garçom clicando em "Confirmar" no mesmo pedido. O comando enviado diz: "mude para `approved` **somente se** ainda estiver `awaiting_approval`". O segundo clique não acha nada para mudar e recebe a mensagem "já foi atualizado em outro aparelho".

O som (alertSound.ts)

O som é gerado pelo próprio navegador (Web Audio API), sem arquivo de áudio. A recepção ouve um "ding-dong" duplo e a cozinha, três notas subindo. Por regra dos navegadores, **nenhum site pode tocar som antes de um clique do usuário**. Por isso existe o botão "Ativar som de alerta", e, depois de recarregar a página, o primeiro clique em qualquer lugar do painel libera o som de novo.

> **Atenção**
> Na operação real, deixe o painel aberto num computador ou tablet **que não entre em descanso de tela**. Com a aba suspensa, o navegador pode atrasar avisos e sons.

O quadro (OrdersBoard)

- **Coluna 1, Aguardando conferência (VHF/Recepção)**: pedidos `awaiting_approval`, com os botões **Confirmar Quarto e Enviar p/ Cozinha** (vira `approved`) e **Recusar** (vira `cancelled`, com confirmação).
- **Coluna 2, Na cozinha (Impressora)**: pedidos `paid` e `approved`, com o selo de pagamento ("Pago · PIX" em verde; "Faturar no quarto" e "Dinheiro" em laranja) e o selo **Impresso / Não impresso**. Os não impressos ficam com borda laranja.
- O título da aba do navegador mostra "(2) Pedidos", para chamar a atenção de longe.

O cupom de 80mm (ReceiptModal)

A impressora EPSON TM-T20X-II usa papel de **80 mm**, com cerca de **72 mm** de área impressa. O cupom é desenhado em CSS com **milímetros** (`width: 80mm`, margens de 4 mm), então a prévia na tela tem o tamanho real do papel. A fonte é monoespaçada (Courier), como a da impressora.

- **Quarto gigante** em fundo preto, para o garçom ler de longe.
- Hóspede, telefone e **CPF mascarado** (`***.982.247-**`), por causa da LGPD.
- Itens, observações em destaque (`>> SEM CEBOLA`), total e forma de pagamento.
- **Faturar no quarto**: caixa "FATURAR NO QUARTO / COLHER ASSINATURA DO HÓSPEDE" com linha para assinar.
- **Dinheiro**: "COBRAR NA ENTREGA / RECEBER R$ X / Levar troco".
- **PIX/cartão pago**: "*** PAGO ONLINE ***".

O modal é desenhado **fora** do `#root`, com um *portal* do React (`createPortal`). Quando o botão chama `window.print()`, uma regra CSS `@media print` esconde o `#root` inteiro e só o cupom vai para o papel. Depois de imprimir, o pedido vira "Impresso".

> **Na prática**
> Na janela de impressão, escolha a **EPSON TM-T20X-II**, papel "Roll Paper 80 mm", margens "Nenhuma" e marque "Gráficos de plano de fundo" (para o quarto sair com o fundo preto).

PARTE IV

Na prática

Fazer funcionar de verdade, investigar problemas e preparar a abertura para hóspedes.

17 Por que hoje aparece “Cardápio indisponível”

O app está conectado ao Supabase? A configuração está certa e a conexão acontece: a URL e a chave estão corretas, e o banco **responde**. Mas ele responde “não”. Testei no dia 22/10/2026:

```
// resposta do Supabase ao pedir as categorias
{ "code": "42501",
  "message": "permission denied for table categories",
  "hint": "Grant the required privileges ... GRANT SELECT ON public.categories TO anon;" }
```

O código **42501** significa “permissão negada”. As tabelas e funções existem (a função `is_admin` respondeu normalmente), mas falta o **GRANT** de leitura. Isso acontece porque o banco ainda tem a **primeira versão** do `schema.sql`, de antes de adicionarmos os GRANTs explícitos para funcionar com a exposição automática de tabelas desligada.

> **Para lembrar**
>
> **A solução é rodar de novo o `supabase/schema.sql` atual** no SQL Editor (próximo capítulo). Não é preciso mudar nada no código React.

Login e senha do admin: ainda não existe nenhum usuário no seu Supabase. Nos meus testes usei um Supabase falso, local, com usuários de teste que só existiam lá e já foram apagados. Você vai criar o usuário de verdade no passo 4 do próximo capítulo.

18 Passo a passo: colocando o teste real para rodar

Faça na ordem. Os nomes dos menus do Supabase podem variar um pouco com o tempo, mas a ideia é sempre a mesma.

Passo 1 · Rodar o schema atualizado

1. Abra `supabase.com`, entre no projeto e clique em **SQL Editor** no menu da esquerda.
2. Clique em **New query**.
3. No VS Code, abra `supabase/schema.sql`, selecione tudo (Ctrl+A), copie e cole no editor.
4. Clique em **Run**. Deve aparecer "Success. No rows returned".

Passo 2 · Rodar o seed

Mesma coisa com `supabase/seed.sql`. Se você já tinha rodado antes, tudo bem: ele não duplica nada. Para conferir, rode:

```
select count(*) from menu_items;   -- deve dar 143
```

Agora rode `npm run dev`: **o cardápio deve aparecer**.

Passo 3 · Ligar o simulador de pagamento (só para testar)

Rode `supabase/dev/payment_simulator.sql` no SQL Editor.

Passo 4 · Criar o login do painel

1. No Supabase: **Authentication → Users → Add user → Create new user**.
2. Digite um e-mail e uma senha forte e marque **Auto Confirm User**. Esse vai ser o seu login no `/admin`.
3. Volte ao SQL Editor e rode, trocando pelo e-mail que você usou:

```
insert into public.admin_users (user_id)
select id from auth.users where email = 'seu-email@exemplo.com';
```

Depois, faça o mesmo para o Cristiano, com o e-mail e a senha dele.

Passo 5 · Desligar o cadastro público

Em **Authentication → Sign In / Providers** (ou "Settings"), desligue **Allow new users to sign up**. Assim, só você cria contas pelo painel do Supabase.

Passo 6 · Liberar "faturar no quarto" (opcional)

Hoje nenhum item aceita faturamento. Para testar, libere por exemplo as carnes e os lanches:

```
update public.menu_items set allow_room_charge = true
where category_id in (select id from public.categories where slug in ('carnes', 'lanches'));
```

Ou faça pelo painel: Editar item → "Aceita cobrança no quarto".

Passo 7 · O roteiro de teste completo

Abra **duas janelas**: uma normal com `http://localhost:5173/RestMenu/admin` (faça login) e uma **anônima** (Ctrl+Shift+N) com `http://localhost:5173/RestMenu/`. A janela anônima garante que o hóspede não está logado como admin. Se o seu terminal mostrar outra porta, use a porta dele.

- ☐ No painel, aba Pedidos ao Vivo: confira o indicador "Ao vivo" e clique em **Ativar som de alerta** (deve tocar um teste).
- ☐ Hóspede: adicione 2 pratos com o + e 1 pelo modal, com observação.
- ☐ Hóspede: Ver pedido → Continuar → preencha tudo → **PIX** → Enviar. Confira que o painel **não** mostra nada ainda.
- ☐ Hóspede: clique em **DEV: Simular Pagamento Aprovado**. O painel deve tocar o som da cozinha e o pedido entra em "Na cozinha".
- ☐ Hóspede: "Fazer novo pedido" → itens liberados → **Faturar no Quarto**. O painel toca o som da recepção e mostra o pedido na coluna 1.
- ☐ Painel: **Confirmar Quarto**. Em até 8 s, a tela do hóspede muda para "Enviado para a cozinha".
- ☐ Painel: **Visualizar / Reimprimir Cupom** → confira o layout → Imprimir (pode ser em PDF) → o selo vira "Impresso".
- ☐ Faça um pedido em **Dinheiro** e **Recuse**: o hóspede vê "Pedido cancelado".
- ☐ Aba Gestão de Cardápio: pause um prato e veja o selo no cardápio do hóspede (recarregue a página dele).

☐ Mude um preço clicando nele. Crie um item com foto. Confira no cardápio.

Na prática

Para ver os pedidos "por dentro", rode no SQL Editor:

```
select order_number, room_number, payment_method, payment_status, total_amount from orders order by created_at desc;
```

19 Como investigar quando algo dá errado

A ferramenta número um é o **DevTools** do navegador (F12):

- **Console**: o app escreve lá os erros, como "Erro ao carregar o cardápio: ...".
- **Network** (Rede): filtre por `supabase`. Clique numa requisição vermelha e veja a aba **Response**: o Supabase sempre explica o motivo em `message`.

Mensagem / código	O que significa	O que fazer
`42501 permission denied for table ...`	Falta GRANT nessa tabela.	Rodar o `schema.sql` atualizado.
`new row violates row-level security policy`	O RLS barrou: quem tentou gravar não tem permissão (por exemplo, não é admin).	Conferir se o usuário está em `admin_users`.
`Invalid login credentials`	E-mail ou senha errados.	Conferir em Authentication → Users.
"Esta conta não tem acesso ao painel"	Login certo, mas `is_admin()` respondeu `false`.	Fazer o `insert into admin_users` do capítulo 18.
`Failed to fetch` / erro de rede	Sem internet, ou a URL do `.env.local` está errada.	Conferir a URL (sem `/rest/v1`) e reiniciar o `npm run dev`.
`Could not find the function ...`	A função não existe no banco.	Rodar o arquivo SQL que cria essa função.
"Um ou mais itens não aceitam cobrança no quarto"	A regra do `allow_room_charge` funcionando.	Liberar o item no painel, se fizer sentido.
Painel em "Reconectando"	O Realtime caiu (rede).	Nada; ele busca de novo a cada 30 s e se reconecta sozinho.

> **Atenção**
> Mudou algo no `.env.local` ou no `.env.development.local`? **Pare e rode o `npm run dev` de novo.** O Vite só lê esses arquivos quando inicia.

20 As fotos do cardápio

O padrão das 3 fotos que existem

- **Costela 12h**: foto de cima (90°, "flat lay"), num prato fundo branco de aba larga, sobre uma mesa lisa marrom-chocolate. Luz natural, suave e difusa, sem sombra dura. Formato quadrado (1:1), prato centralizado ocupando uns 85% da imagem.
- **Hambúrgueres**: ângulo de uns 45°, numa tábua de madeira em dois tons. Ao fundo, desfocados, um guardanapo branco, um ramequim de molho e copos de bebida. Luz de janela, cores quentes.

Em resumo: fotos **reais** do restaurante, com luz natural, louça branca, madeira e marrom, sem fundo artificial e sem exagero de edição.

Por que eu não gero essas fotos

Eu não tenho ferramenta de geração de imagens. Além disso, existe um cuidado importante: uma foto inventada por IA mostra um prato que **não é o que a cozinha serve** (outra porção, outro empratamento, outro acompanhamento). Num cardápio de restaurante, o hóspede espera receber o que viu, e uma diferença grande gera reclamação e pode ser vista como propaganda enganosa.

O caminho recomendado

1. **Fotografar os pratos de verdade**, repetindo a receita acima: perto de uma janela (luz indireta, sem flash), mesa marrom ou tábua de madeira, prato branco, celular paralelo à mesa (pratos fundos) ou a 45° (lanches). Tire várias fotos e escolha a melhor.
2. Se quiser, use IA **só para tratar** a foto real (luz, cor, fundo), como parecem ter sido tratadas as 3 atuais.
3. Suba pelo painel: Editar item → Adicionar foto. O app reduz o tamanho, envia e liga o ícone de câmera sozinho.

> **Na prática**
> Se for usar uma IA de imagem (o Gemini, por exemplo) para **tratar** uma foto real, um pedido-modelo é: "Mantenha exatamente o prato e a porção desta foto. Ajuste para luz natural suave de janela, prato branco sobre mesa lisa marrom-chocolate, vista de cima, formato quadrado, estilo fotografia de restaurante sofisticado, sem elementos novos."

Quanto às fotos que faltam: dos 143 itens, 61 apontam para um caminho de foto, mas só 3 desses arquivos existem. Os outros 58 mostram a imagem padrão até receberem foto pelo painel.

21 Antes de abrir para hóspedes de verdade

- ☐ Rodar `supabase/dev/remove_payment_simulator.sql`. **Obrigatório**: com a função de simulação no banco, qualquer pessoa "paga" o próprio pedido sem dinheiro.
- ☐ Integrar um **gateway de pagamento** real (Mercado Pago, Asaas, Pagar.me...). A confirmação deve chegar por **webhook** numa *Edge Function* do Supabase, nunca pelo navegador.
- ☐ Abrir a conta do gateway no **CNPJ do restaurante**, para o dinheiro cair direto para ele.
- ☐ Criar o **programa de impressão** no computador da cozinha, que escuta os pedidos e imprime sozinho na Epson. Hoje a impressão é pelo botão (o "backup").
- ☐ Garantir que o cadastro público está desligado (capítulo 18, passo 5).
- ☐ Revisar o `allow_room_charge` de cada item com o Cristiano.
- ☐ Apagar os pedidos de teste (os pagos pelo simulador têm `payment_provider_id` começando com `SIMULADO-`).
- ☐ Publicar uma política de privacidade simples (LGPD), já que o app guarda nome, CPF e telefone.
- ☐ Fotografar os pratos (capítulo 20).
- ☐ Fazer o commit e o deploy (`npm run deploy`).

Coisas que ainda não existem no painel: excluir item, reordenar itens e categorias, e relatórios de vendas.

22 Glossário

Termo	Explicação
Banco de dados	Onde os dados ficam guardados de forma organizada e permanente, num servidor.
Supabase	Serviço na nuvem que oferece banco (Postgres), login, arquivos e tempo real, sem precisar de servidor próprio.
Postgres / PostgreSQL	O tipo de banco de dados que o Supabase usa.
SQL	A linguagem para ler e alterar dados do banco.
Tabela, linha, coluna	Planilha, registro e campo.
Chave primária	A coluna que identifica cada linha de forma única (o `id`).
Chave estrangeira	Uma coluna que aponta para o `id` de outra tabela.
Schema	A estrutura do banco: quais tabelas, colunas e regras existem.
Seed	Os dados iniciais usados para preencher o banco.
Query	Uma consulta ou comando enviado ao banco.
Join	Juntar duas tabelas pela coluna que as liga.
GRANT	Permissão por tabela: quem pode ler, inserir, alterar ou apagar.
RLS / policy	Regras por linha: "esta pessoa pode ver ou alterar ESTA linha?".
anon / authenticated	Visitante sem login / visitante com login.
RPC	Uma função do banco que o app chama pelo nome.
security definer	Função que roda com a permissão do dono do banco, mas só faz o que está escrito nela.
Transação	Um conjunto de comandos que acontece inteiro ou não acontece (nada fica pela metade).
UUID	Um código aleatório e impossível de adivinhar, como `8368db83-...`.
JSON / jsonb	Um objeto no formato do JavaScript, guardado numa coluna.
Auth / sessão / JWT	O login. A sessão é o "crachá" digital (um token JWT) que prova quem é a pessoa.
Chave publishable / secret	A pública pode ir no site; a secreta (service_role) ignora todas as regras e nunca pode ir para o site.
Storage / bucket	Armazenamento de arquivos / uma "pasta" dele.
Realtime / WebSocket	Conexão que fica aberta para o servidor avisar o navegador na hora.
Polling	Perguntar ao servidor de tempos em tempos ("mudou algo?").
Webhook	Quando um serviço externo (o gateway) avisa o seu sistema chamando um endereço dele.
Gateway de pagamento	A empresa que processa PIX e cartão (Mercado Pago, Asaas...).
Edge Function	Um pedacinho de código que roda nos servidores do Supabase, e não no navegador.
Hook	Uma função do React que começa com `use` e guarda lógica reaproveitável.
Context	Dados compartilhados por vários componentes sem passar props.
Portal	Desenhar um componente React fora da div principal (`#root`).
Atualização otimista	Mudar a tela antes da resposta do servidor e desfazer se der erro.
Lazy loading	Baixar um pedaço do código só quando ele é necessário.
Build / deploy	Gerar a versão final do site / publicá-la.
LGPD	A lei brasileira de proteção de dados pessoais.

23 Exercícios para fixar

Os exercícios de SQL abaixo **só leem dados** (`select`), então pode rodar no SQL Editor sem medo. Tente fazer antes de olhar a resposta.

1. Liste o nome e o preço dos 5 itens mais caros do cardápio.

   Resposta: `select name, price from menu_items order by price desc limit 5;`

2. Quantos itens estão pausados agora?

   Resposta: `select count(*) from menu_items where is_available = false;`

3. Mostre cada vinho tinto de Malbec com o preço.

   Resposta: `select i.name, i.price from menu_items i join categories c on c.id = i.category_id where c.slug = 'malbec';`

4. Quais são as abas do cardápio (categorias sem pai)?

   Resposta: `select name from categories where parent_id is null order by sort_order;`

5. Mostre os pedidos de hoje com quarto, status e total.

   Resposta:
   `select order_number, room_number, payment_status, total_amount from orders where created_at::date = current_date order by created_at;`

6. Qual é o nome do prato em inglês do item com slug `cn2`?

   Resposta: `select translations→'en'⟶'name' from menu_items where slug = 'cn2';` (a seta `→` entra no JSON e a `⟶` devolve o texto).

7. *(React)* Em que arquivo você mudaria o texto "Indisponível no momento" em espanhol?

   Resposta: `src/locales/es.json`, na chave `menu.unavailable`.

8. *(React)* O hóspede reclama que o carrinho "sumiu" depois de fechar o navegador. Onde ele é salvo?

   Resposta: no `localStorage` do aparelho, chave `irius:cart` (`src/cart/CartProvider.tsx`). Numa janela anônima, ele some ao fechar, e isso é normal.

9. *(Pensar)* Por que o front não manda o preço para `create_safe_order`?

   Resposta: porque qualquer pessoa pode alterar o que o navegador envia. O banco busca o preço oficial e calcula o total sozinho.

10. *(Pensar)* O que acontece se você esquecer de rodar `remove_payment_simulator.sql` antes de abrir o app?

    Resposta: qualquer pessoa pode chamar `dev_simulate_payment` para o próprio pedido PIX e ele vai para a cozinha como pago, sem dinheiro nenhum.

Para terminar

Você não precisa dominar SQL para manter este app no dia a dia: quase tudo se faz pelo painel. O SQL aparece em três momentos: na instalação (capítulo 18), em consultas de conferência (exercícios acima) e quando o sistema ganhar recursos novos. Com este guia, você já sabe ler os arquivos e entender o que cada linha faz.